

UNIVERSITÀ DEGLI STUDI DI UDINE

DIPARTIMENTO DI SCIENZE MATEMATICHE,
INFORMATICHE E FISICHE

IBML - Internet of Things, Big Data, Machine Learning



Internet of Things Progetto Pet Tracker

Autori

Andrea Gioia, 169484
Alessio Nicodemo, 166972
Mattia Nadal, 167372
Francesco Sabot, 167339

169484@spes.uniud.it
166972@spes.uniud.it
167372@spes.uniud.it
167339@spes.uniud.it

Sommario

Il progetto Pet Tracker propone la realizzazione di un sistema per il tracciamento in tempo reale di animali domestici di taglia media o grande, tramite un dispositivo *embedded* indossabile. L'obiettivo del lavoro è sviluppare una soluzione affidabile, scalabile e a basso consumo per il monitoraggio continuo dell'animale.

La soluzione si basa su una piattaforma ESP32-S3 con connettività LTE e modulo GNSS, in grado di acquisire e trasmettere lo stato della batteria del dispositivo, insieme alla posizione geografica e all'attività motoria dell'animale, verso un *backend* remoto. L'architettura del sistema segue il modello *IoT World Forum* (IoTWF) e integra diverse tecniche di ottimizzazione energetica, tra cui l'utilizzo delle modalità *deep sleep* e *hot start*. I dati raccolti vengono elaborati dal *backend*, resi accessibili tramite API REST e visualizzati attraverso un'applicazione mobile dedicata.

Indice

1	Hardware	4
1.1	LilyGo T-SIM7670G-S3	4
1.2	Antenne LTE e GNSS	4
1.3	Batteria e power Management	4
1.4	BMA400 CJMCU-400	4
2	Firmware: logica di funzionamento	4
2.1	Gestione della persistenza e hot start	4
2.2	Protocollo di comunicazione HTTPS	5
2.3	Risparmio energetico	5
2.4	Acquisizione GPS e timeout	6
2.5	Connessione alla rete LTE	6
3	Backend e database	6
3.1	Architettura di hosting e operational continuity	6
3.2	Modellazione dei dati e API rules	7
3.3	Pulizia e Normalizzazione (Server-side Hooks)	7
3.4	Macchina a Stati delle Attività e Logiche di Isteresi	8
3.5	Lifecycle del deep sleep e servizi temporali	9
3.5.1	Gestione del risveglio e anti-spuria	9
3.5.2	Watchdog di Inattività e Split Notturmo	10
4	Front-end e Applicazione Mobile	10
4.1	Architettura a livelli	10
4.2	Autenticazione e persistenza della sessione	11
4.3	Splash Screen e Caricamento Parallelo	11
4.4	Comunicazione in Tempo Reale	11
4.5	Geofencing e Geocoding	12
4.6	Schermate e Funzionalità Principali	12
4.6.1	Analisi delle Attività	12
4.6.2	Mappe in Tempo Reale	13
4.6.3	Gestione delle Zone Sicure	13
5	Sistema di Notifiche Push	14
5.1	Analisi dei Vincoli Tecnologici: Incompatibilità Goja e Firebase	14
5.2	Architettura del FCM Bridge	15
5.3	Logica di Invio e Trigger	15
5.4	Gestione permessi	15
6	Vincoli di Sistema	16
7	Implementazioni Future	16
7.1	Stato della batteria	17
7.2	Database, Infrastruttura Cloud e Gestione della Concorrenza	17
7.3	Ingegnerizzazione e Costruzione della Scheda Hardware	17
8	Conclusioni	18

1 Hardware

1.1 LilyGo T-SIM7670G-S3

Il dispositivo utilizza una scheda LilyGo T-SIM7670G-S3 [1] basata sul microcontrollore ESP32-S3 dual-core. Quest'ultimo gestisce la logica applicativa e la comunicazione con il modem SIM7670G tramite interfaccia UART.

1.2 Antenne LTE e GNSS

Il sistema utilizza un'antenna flessibile LTE [2] per la connettività dati e un'antenna ceramica attiva [3] per il ricevitore GNSS integrato, volta a garantire la precisione della localizzazione *outdoor*.

1.3 Batteria e power Management

Il sistema è alimentato da una batteria ricaricabile agli ioni di litio da 3000 mAh e 30 A. Il monitoraggio della carica avviene tramite un partitore di tensione collegato al pin ADC_BAT (GPIO 4), per calcolarne la percentuale residua.

1.4 BMA400 CJMCU-400

L'accelerometro utilizzato è un BMA400 [4] integrato sulla *breakout board* CJMCU-400. Il sensore viene interrogato per rilevare il movimento, consentendo l'implementazione di un contapassi utile al monitoraggio dell'attività motoria dell'animale.

2 Firmware: logica di funzionamento

Il firmware, sviluppato in ambiente Arduino per il *System on Chip* (SoC) ESP32-S3, gestisce l'acquisizione dei dati dai sensori, il controllo del modem LTE e l'ottimizzazione energetica tramite una macchina a stati finiti.

2.1 Gestione della persistenza e hot start

Per ridurre drasticamente il *Time To First Fix* (TTFF) del modulo GNSS, il firmware sfrutta la memoria *Real-Time Clock* (RTC) dell'ESP32. Tale area di memoria consente di preservare i dati critici anche durante i cicli di *deep sleep*. Le variabili critiche, come l'ultima posizione valida (`lastLat`, `lastLon`) e i timestamp (`lastGpsDate`, `lastGpsTime`), vengono dichiarate con l'attributo `RTC_DATA_ATTR`.

Al risveglio del dispositivo, la funzione `iniettaGps()` esegue le seguenti operazioni:

- Recupera le coordinate geografiche e temporali salvate nel ciclo precedente;
- Inietta la posizione nota nel modulo GNSS tramite il comando `AT+CGNSSPOS`;
- Sincronizza l'orario del modem con il comando `AT+CGNSSTIME`.

Questo meccanismo abilita la modalità *hot start*, che minimizza il tempo di ricerca dei satelliti e, di conseguenza, il consumo energetico del sistema.

2.2 Protocollo di comunicazione HTTPS

La trasmissione dei dati al *backend* PocketBase è affidata a una serie di comandi AT inviati via seriale al modem SIM7670G. Il firmware incapsula le informazioni in un oggetto JSON strutturato, trasmesso tramite una richiesta HTTPS POST. Il *payload* include:

- **board_id**: un identificativo univoco derivato dall'IMEI del modem;
- **geo**: un oggetto con latitudine e longitudine per la gestione cartografica;
- **battery / battery_percent / charging**: parametri relativi allo stato di carica della batteria;
- **steps**: conteggio dei passi rilevato dall'accelerometro nella sessione corrente;
- **sleep**: flag booleano che segnala al server l'imminente entrata in stato di inattività;
- **trip**: flag booleano che segnala al server che l'animale si trova su un veicolo in movimento. Quando attivo, ha priorità assoluta sulla logica di *geofencing* e sopprime il *deep sleep* per garantire la continuità del tracciamento.

La sequenza di invio si avvale dei comandi AT+HTTPINIT, AT+HTTPPARA e AT+HTTPDATA per configurare la sessione HTTP, inserire i dati e infine avviare la trasmissione con AT+HTTPACTION=1. Al termine di ogni invio, la sessione viene chiusa con AT+HTTPTERM per liberare le risorse del modem.

2.3 Risparmio energetico

Per massimizzare l'autonomia, la gestione energetica viene modulata in base alla rilevazione del movimento e allo stato di viaggio:

1. **Rilevamento Attività**: il firmware monitora costantemente il numero di passi, azzerando a ogni nuovo movimento un timer di inattività (`lastActivityTime`).
2. **Modalità Viaggio**: se il flag **trip** è attivo (velocità GPS > 10 km/h), il *deep sleep* viene disabilitato indipendentemente dall'inattività rilevata dall'accelerometro, garantendo la trasmissione dei dati a intervalli regolari durante il viaggio.
3. **Ibernazione**: se non viene rilevata attività per un tempo superiore a `SLEEP_TIMEOUT` (30 secondi) e il flag **trip** non è attivo, il sistema invia un ultimo pacchetto dati aggiornando il flag **sleep** a `true`, entrando in *deep sleep*. Vengono quindi spenti i moduli radio tramite i comandi `AT+CGNSSPWR=0` e `AT+CPOWD=1`. Il risveglio è gestito tramite interrupt hardware sul pin `GPIO_5`, generato dall'accelerometro solo al rilevamento di un nuovo movimento.

Inoltre, il firmware integra specifiche strategie di controllo energetico volte a eliminare i consumi statici e a prevenire alterazioni nelle letture dei sensori:

- **Controllo ADC**: il circuito del partitore di tensione per la batteria viene attivato solo durante la lettura tramite il pin `BAT_ADC_EN`, garantendo la precisione del voltaggio acquisito dal pin `ADC_BAT`. Il valore grezzo viene mediato su 10 campionamenti consecutivi e scalato con fattore pari a 2 per compensare il partitore. La percentuale di carica è calcolata per interpolazione lineare nell'intervallo compreso tra 3,4 V e 4,2 V, corrispondenti rispettivamente a 0% e a 100% di carica.

- **Stato USB:** dal registro hardware `USB_SERIAL_JTAG_EP1_CONF_REG` dell'ESP32, il sistema rileva se il dispositivo è collegato a una fonte di ricarica USB, impostando dinamicamente il campo `charging` nel pacchetto dati. In stato di ricarica, tensione e percentuale non vengono aggiornate per evitare misurazioni alterate dalla corrente in ingresso, vengono invece restituiti gli ultimi dati validi salvati nella memoria RTC.

2.4 Acquisizione GPS e timeout

Il modulo GNSS viene interrogato tramite il comando `AT+CGNSSINFO`. La risposta viene analizzata campo per campo, estraendo latitudine, longitudine e velocità in formato decimale, evitando la necessità di conversione dal formato NMEA gradi-minuti. Il firmware attende fino a un massimo di `GPS_TIMEOUT` (180 secondi) per ottenere un fix valido; superato tale limite senza una posizione acquisita, il ciclo corrente viene saltato e il sistema ritenta al loop successivo. Una volta ottenuto un fix valido, le coordinate e il timestamp vengono salvati in memoria RTC per abilitare l'*hot start* al ciclo successivo.

2.5 Connessione alla rete LTE

All'avvio, il firmware tenta di registrarsi alla rete LTE attraverso la funzione `connectToNetworkFast()`. Vengono configurati la modalità radio (`AT+CNMP=38` per LTE only), il profilo APN dell'operatore e l'attivazione del contesto dati con `AT+CNACT=0,1`. La registrazione viene verificata in polling tramite il comando `AT+CREG?`, controllando la presenza dei codici di stato 0,1 (registrato in rete home) o 0,5 (roaming). Se entro `NET_TIMEOUT` (60 secondi) la registrazione non viene completata, il dispositivo entra direttamente in *deep sleep* per evitare consumi inutili in assenza di copertura.

3 Backend e database

La gestione della persistenza e della modellazione dei dati è basata su PocketBase [5], una soluzione open-source strutturata su SQLite scelta per la sua reattività, il ridotto consumo di risorse e il supporto di un'interfaccia API RESTful. La logica applicativa lato server e l'integrazione dei flussi di dati si sviluppano estendendo il nucleo del framework direttamente nel file principale di runtime `main.pb.js`, che gestisce sia gli eventi reattivi, noti come *hooks*, sia i servizi temporali pianificati, definiti *cron jobs*. Poiché il runtime JavaScript di PocketBase si basa sull'interprete Goja incorporato in Go, l'intera esecuzione del codice avviene in modalità *single-threaded*, condividendo il medesimo spazio d'esecuzione globale e lo stesso ciclo di eventi del server.

3.1 Architettura di hosting e operational continuity

La gestione operativa dell'infrastruttura di backend è stata ottimizzata per garantire la disponibilità del dato, la tolleranza ai guasti e il monitoraggio dei flussi durante la fase di prototipazione e test:

- **Network Exposure (NAT Traversal):** l'istanza è ospitata su una *workstation locale* e data la necessità di connettersi al backend sia dal dispositivo (tramite rete cellulare LTE) sia dall'applicazione mobile (tramite rete dati/Wi-Fi), il vincolo del *NAT Traversal* e l'assenza di un IP pubblico statico sulla rete locale sono stati

superati mediante l'impiego di ngrok [6]. Questa tecnologia di *tunneling* crea un canale sicuro HTTPS fungendo da *reverse proxy*, reindirizzando il traffico in ingresso verso l'istanza locale di PocketBase e garantendo l'accessibilità dei servizi in tempo reale.

- **Strategia di Backup a Caldo (Data Retention):** per garantire la resilienza dei dati e prevenire perdite accidentali dovute a guasti del server locale, è stato strutturato un sistema di backup automatizzato tramite l'utility rclone [7], uno strumento a riga di comando per la gestione del cloud storage. Il processo è configurato per eseguire la sincronizzazione incrementale della directory dei dati di PocketBase verso un *bucket remoto* su Backblaze B2 [8] tramite un *cron job* a cadenza mensile.
- **Monitoraggio Esterno e Uptime:** l'accessibilità e lo stato di salute del sistema sono vigilati esternamente tramite UptimeRobot [9]. Dato che l'accessibilità del backend dipende dalla stabilità del tunnel ngrok, il monitoraggio interroga l'endpoint HTTPS a intervalli regolari, isolando tempestivamente le anomalie di rete e inviando notifiche immediate in caso di interruzione del tunnel o del server locale.

3.2 Modellazione dei dati e API rules

I dati inviati dalla board transitano nella collezione di staging `data_sent_raw`, la quale funge da punto di ingresso grezzo ed isola il database da potenziali fluttuazioni della rete cellulare. La struttura del record prevede i seguenti campi tipizzati:

- **timestamp:** Riferimento temporale hardware dell'acquisizione.
- **lat / lon:** Coordinate geografiche acquisite dal modulo GNSS.
- **battery / battery_percent:** Stato di carica dell'accumulatore espresso in voltaggio e percentuale.
- **charging:** Variabile booleana indicante se la batteria è in corso di ricarica.
- **steps:** Conteggio lineare dei passi rilevato dall'accelerometro BMA400.
- **sleep:** Flag booleano indicante se il dispositivo è in stato di *deep sleep*.
- **trip:** Flag booleano indicante se l'animale si trova su un veicolo in movimento.

L'accesso alle collezioni memorizzate nel backend avviene esclusivamente tramite l'interfaccia RESTful API fornita nativamente, separando nettamente il livello di persistenza dal livello di fruizione mobile dell'applicazione Flutter.

3.3 Pulizia e Normalizzazione (Server-side Hooks)

L'elaborazione dei messaggi IoT segue un pattern *Staging-and-Purge* sequenziale gestito lato server tramite i *JavaScript Event Hooks* nativi del sistema. L'hook `onRecordAfterCreateSuccess` si attiva immediatamente dopo il consolidamento con successo di ogni pacchetto grezzo nella collezione `data_sent_raw`. Per ottimizzare le performance computazionali ed eliminare query ridondanti nello stesso ciclo, il sistema esegue una **lettura centralizzata** della configurazione della scheda (`getBoardRecord`) all'inizio dell'esecuzione dell'hook, distribuendo il record ottenuto come parametro a tutte le funzioni della pipeline. Il trigger elabora ogni pacchetto in quattro fasi sequenziali:

1. **Batteria:** Salva il record nella collezione verticale `battery_data` e verifica se inviare notifiche push di cambio soglia o allerta energetica.
2. **Activity Tracking:** Delega l'analisi della telemetria al modulo `activity_manager.js`, che aggiorna la macchina a stati e gestisce la persistenza delle sessioni.
3. **Posizioni:** Salva le coordinate GPS nella collezione `positions` agganciandole all'attività determinata al punto precedente, a condizione che i dati GNSS siano validi e l'attività non sia stata rigettata.
4. **Eliminazione dato grezzo:** Rimuove permanentemente il record originario da `data_sent_raw` all'interno del blocco `finally` dell'hook. Questo costrutto non serve a garantire l'atomicità transazionale (poiché l'evento scatta a inserimento già consolidato), ma funge da garante di svincolo per l'ottimizzazione dello spazio fisico: assicura la cancellazione fisica del record di staging e previene il rigonfiamento del database SQLite anche qualora una delle funzioni logiche precedenti (es. il modulo attività o il salvataggio della posizione) dovesse lanciare un'eccezione non gestita a runtime.

3.4 Macchina a Stati delle Attività e Logiche di Isteresi

Il cuore del modulo `activity_manager.js` è una macchina a stati che modella il comportamento cinematico e geografico dell'animale. Gli stati sono suddivisi in *attivi* (dispositivo sveglio) e *sleep* (dispositivo in *deep sleep*), legati da una corrispondenza biunivoca riassunta nella Tabella 1.

Attivi	Sleep	Allarme	Geofence	Descrizione
i (inside)	d	Any	True	Animale all'interno del perimetro sicuro.
v (trip)	a	False	False	In viaggio su veicolo, allarme spento.
r (trip)	q	True	False	In viaggio su veicolo, allarme attivo (critico).
w (walk)	z	False	False	Fuori perimetro in passeggiata, allarme spento.
s (search)	p	True	False	Fuori perimetro, allarme attivo (emergenza).

Tabella 1: Macchina a stati delle attività: corrispondenze tra stati attivi e sleep.

Tutti gli stati sleep si comportano in modo uniforme: vengono sempre archiviati come sessioni storiche chiuse impostando il flag `is_active = false`. Prima di eseguire qualsiasi valutazione di business logic, la funzione `computeStatus` normalizza internamente gli stati sleep convertendoli nel rispettivo equivalente attivo (tramite il dizionario `SLEEP_TO_ACTIVE`, es. "a" → "v" e "q" → "r"). Questo garantisce che la logica di analisi riconosca da quale contesto provenga l'animale anche dopo lunghe fasi di inattività hardware. Al fine di minimizzare la granularità dei record sul DBMS e ottimizzare le query storiche, il sistema applica un principio di **estensione temporale**: se un pacchetto conferma lo stato corrente, l'activity manager si limita ad aggiornare il timestamp di fine evento (`end_time`) e a incrementare il contatore lineare dei passi (`total_steps`). La creazione di una nuova riga nel database (con la contestuale chiusura permanente della

precedente) avviene esclusivamente in presenza di un'effettiva transizione di stato. Una transizione tra "v" e "r" indotta dal cambio manuale del flag di allarme da parte dell'utente forza la chiusura e la riapertura immediata della sessione, garantendo la tracciabilità del cambio di contesto e la variazione del livello di urgenza delle notifiche. Le transizioni tra gli stati sono governate da un algoritmo a priorità decrescente:

1. **Priorità 1 (Massima) - Stato Viaggio:** Se il flag `trip = true` e il contatore dei passi è nullo (`steps == 0`), lo stato viene forzato a "v" o "r" a seconda dell'allarme, azzerando i falsi positivi derivanti da movimenti muscolari isolati dell'animale o rumore dell'accelerometro.
2. **Priorità 2 - Memoria del Viaggio:** Qualora il sensore hardware rilevi il termine del viaggio (`trip = false`) ma l'animale non ha ancora computato passi (`steps == 0`), il sistema applica una memoria di stato: assume che il veicolo sia momentaneamente fermo (es. a un semaforo o in sosta con l'animale a bordo) e preserva lo stato *Trip* corrente ("v" o "r"). Il ricalcolo geografico pulito si attiva solo se `steps > 0`, ovvero al primo movimento muscolare effettivo.
3. **Priorità 3 - Analisi Geofence:** Se l'animale si trova all'interno delle coordinate del cerchio sicuro configurato, lo stato viene impostato a "i" (Inside).
4. **Priorità 4 (Minima) - Stato Esterno:** Analisi combinata delle coordinate esterne e dell'allarme utente, che determina la transizione in passeggiata ordinaria ("w") o nello stato critico di ricerca ("s").

3.5 Lifecycle del deep sleep e servizi temporali

3.5.1 Gestione del risveglio e anti-spuria

Al risveglio dal *deep sleep*, il tracker trasmette un pacchetto privo di una sessione attiva aperta sul database, in quanto la precedente era stata chiusa all'atto dell'addormentamento. Il sistema interroga la collezione escludendo i record anomali ed estrae la sessione chiusa più recente (`recentClosed`) per ricavare il `prevStatus` corretto da passare a `computeStatus`.

Il sistema gestisce il risveglio secondo due scenari logici:

- **Risveglio nello stesso stato:** Se lo stato attivo calcolato coincide con lo stato attivo di partenza dello sleep precedente (es. dormiva in casa "d" normalizzato a "i", e il nuovo stato è ancora "i"), la sessione viene riattivata in continuità impostando `is_active = true` senza limiti temporali, evitando frammentazioni e collegando le attività attraverso i cicli di sonno.
- **Risveglio con cambio di stato:** La sessione chiusa viene sanata convertendo lo status nella sua versione attiva originaria a fini di coerenza storica (es. "d" → "i"), e viene parallelamente istanziata una nuova attività con il nuovo stato calcolato. Le sessioni chiuse dal watchdog con `anomaly = true` sono tassativamente escluse da questo flusso e non vengono mai riattivate.

Per ottimizzare l'efficienza del thread unico di esecuzione, il modulo implementa un **filtraggio delle notifiche spurie** (*STEP 3 anticipato*): se viene ricevuto un pacchetto di quiete (`sleep=true`) in assenza di una sessione attiva sul database, l'inserimento viene

intercettato e scartato preventivamente a livello logico, impedendo la proliferazione di record ridondanti e l'invio di catene di notifiche push duplicate all'utente.

3.5.2 Watchdog di Inattività e Split Notturmo

La coerenza temporale delle sessioni e l'allineamento dei calendari storici sono garantite da due cron job registrati all'avvio nel thread principale dell'applicazione:

- **Watchdog (ogni minuto):** Isola le sessioni attive che non trasmettono dati entro un intervallo stabilito (10 minuti), causate da anomalie impreviste (es. perdita di segnale LTE o spegnimento repentino). Il sistema forza la chiusura del record impostando `is_active = false` e l'attributo `anomaly = true`, notificando istantaneamente gli utenti della board. Il watchdog esclude per design le sessioni in stato sleep, le quali possono rimanere legittimamente inattive a lungo senza costituire un'anomalia.
- **Split Notturmo Calendario (23:59 ora italiana):** Fraziona le sessioni attive a cavallo di due giornate per non alterare le statistiche e i grafici storici giornalieri dell'utente. I timestamp vengono memorizzati nativamente in UTC, calcolando dinamicamente l'offset stagionale europeo (ora legale CEST a 21:59:59 UTC, ora solare CET a 22:59:59 UTC) tramite la funzione helper `getItalyTime()`, azzerando l'uso di costanti rigide o offset hardcoded. Il cron è schedulato su entrambi gli orari UTC critici (59 21,22 * * *) e un controllo interno filtra il tick non pertinente alla giornata corrente. Le sessioni in sonno vengono risvegliate forzatamente prima del frazionamento per garantire l'integrità del riepilogo storico, mentre le sessioni già marcate come anomale dal watchdog vengono ignorate e non duplicate nel giorno successivo.

4 Front-end e Applicazione Mobile

L'applicazione mobile è stata sviluppata in *Flutter* [10], un framework cross-platform che consente di produrre un'unica base di codice per Android e iOS. I test sono stati condotti **esclusivamente su ambiente Android** a causa di vincoli tecnici, tra cui l'assenza di postazioni di lavoro con macOS, requisito necessario per il processo di build in ambiente Apple, e la mancanza di una licenza *Apple Developer Program*, indispensabile per garantire la persistenza dell'applicazione su dispositivo fisico oltre il limite di sette giorni previsto per i profili gratuiti. La comunicazione con il backend avviene tramite il PocketBase SDK ufficiale per Dart, che gestisce le chiamate REST e i meccanismi di autenticazione.

4.1 Architettura a livelli

Il codice è strutturato in quattro livelli principali, per separare le diverse responsabilità:

- **Repositories:** accesso ai dati remoti tramite PocketBase SDK, fornendo alle schermate dati in tempo reale e pronti all'uso.
- **Models:** definiscono la struttura dei dati convertendo i record JSON del backend in oggetti Dart.
- **Services:** gestiscono le funzionalità di sistema e le integrazioni esterne (autenticazione, mappe, tracciamento GPS e notifiche).

- **Screens:** interfaccia utente che si limita a richiedere i dati ai repository e a reagire ai loro aggiornamenti.

Questa struttura garantisce un codice ordinato e facile da aggiornare, mantenendo la logica di funzionamento ben separata dall'interfaccia grafica.

4.2 Autenticazione e persistenza della sessione

Per permettere all'utente di restare collegato in modo permanente e sicuro, abbiamo esteso l'`AuthStore` nativo di PocketBase, che normalmente salva i dati solo temporaneamente in memoria. L'autenticazione è quindi gestita tramite un `SecureAuthStore` personalizzato che agisce come una *cassaforte digitale*, riceve il token (JSON Web Token) dal server al momento del login e lo archivia in forma cifrata all'interno del dispositivo tramite `flutter_secure_storage`. Questo sistema sfrutta i sistemi di protezione hardware del dispositivo (Keystore su Android e Keychain su iOS) per impedire accessi non autorizzati.

Al riavvio dell'app, il token viene riletto dallo storage sicuro e viene tentato un refresh automatico della sessione. In caso di successo, l'utente accede direttamente alla schermata principale senza dover reinserire le credenziali; in caso di token scaduto o non valido, lo store viene svuotato e l'utente viene reindirizzato alla schermata di login.

4.3 Splash Screen e Caricamento Parallelo

La schermata di avvio svolge un ruolo attivo nel precaricare i dati necessari alla home page. Una volta verificata l'autenticazione, vengono avviate in parallelo, le seguenti operazioni:

- recupero del profilo utente e dello stato dell'allarme;
- recupero dell'ultima posizione GPS registrata dal dispositivo;
- recupero delle attività del giorno corrente filtrate per `board_id`.

Il `board_id`, identificativo univoco della board LilyGo associata a un dispositivo per utente, viene risolto dinamicamente interrogando la collezione `boards` sul backend tramite repository. Questo permette all'app di trovare il dispositivo corretto automaticamente, evitando di dover inserire il codice manualmente nel software.

4.4 Comunicazione in Tempo Reale

Completata la fase iniziale di caricamento dati, l'app stabilisce una **connessione persistente** al database, passando da una visualizzazione statica ad un monitoraggio dinamico.

Per le funzionalità che richiedono aggiornamenti continui come la posizione GPS, l'app sfrutta il meccanismo di *subscribe* offerto da PocketBase, basato su `Server-Sent Events`. I repository (come `PositionsRepository`) espongono uno `Stream` tipizzato (ad esempio `Stream<Positions>`) a cui le schermate si sottoscrivono tramite una `StreamSubscription`.

In questo modo, ogni nuovo record inserito nel database viene notificato istantaneamente all'app. L'interfaccia utente si aggiorna dinamicamente evitando la tecnica del *polling*, che costringerebbe l'app a inviare continue richieste al server, ed eliminando la necessità di qualsiasi ricaricamento manuale della pagina da parte dell'utente.

Per garantire l'efficienza dell'applicativo e prevenire *memory leak*, ogni sottoscrizione viene rigorosamente interrotta tramite il comando `cancel()` nel metodo `dispose()` quando l'utente naviga verso un'altra schermata. Questa accortezza evita di mantenere attive connessioni inutili in background, ottimizzando l'uso della batteria e del traffico dati.

4.5 Geofencing e Geocoding

La logica di geofencing è implementata anche dal lato client tramite algoritmo *Ray-Casting*. Dato un punto GPS e un poligono definito da una lista di vertici, l'algoritmo traccia un raggio orizzontale dal punto verso l'infinito e conta le intersezioni con i lati del poligono, se il numero è dispari indica che il punto è interno al poligono, altrimenti è esterno. Questa implementazione sul dispositivo evita un sovraccarico di rete verso il server e permette di gestire le funzionalità visive in tempo reale, in totale autonomia dal *backend*.

Le zone vengono recuperate dalla collezione `geofences` su PocketBase, dove ogni record memorizza il nome della zona, i dati dell'indirizzo e la lista dei vertici del poligono. Solo le zone con il flag `is_active` impostato a `true` vengono considerate nel calcolo. Il risultato della verifica è una stringa con il nome della zona di appartenenza, oppure *"Fuori zona sicura"* se il punto non rientra in nessuna area attiva.

L'app integra un servizio di **geocoding** basato sulle API di *Nominatim* [11], un motore di ricerca che interroga il database geografico open-source di OpenStreetMap per consentire:

- **Reverse Geocoding**: tradurre le coordinate GPS in un indirizzo testuale completo (Via, Civico, CAP e Città) per facilitare la lettura all'utente;
- **Forward Geocoding**: consentire all'utente di creare nuove zone inserendo un indirizzo testuale, che il sistema converte automaticamente in coordinate geografiche per posizionare correttamente il poligono sulla mappa.

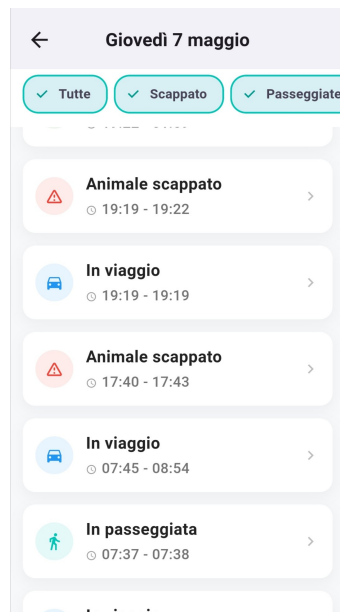
4.6 Schermate e Funzionalità Principali

4.6.1 Analisi delle Attività

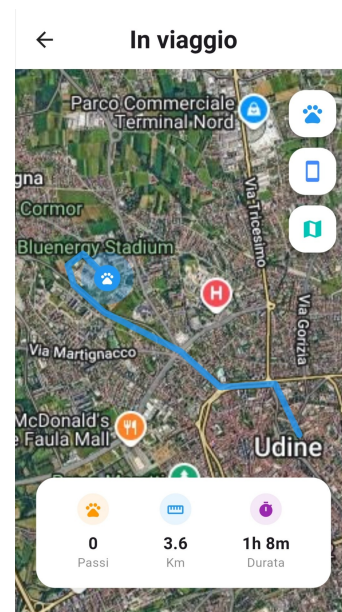
L'architettura dell'applicazione ruota attorno alla dashboard principale in `home.dart`, che funge da centro di controllo dinamico per il monitoraggio dell'animale, mostrandone l'ultima posizione rilevata. Un'altra caratteristica importante della pagina è l'interruttore dell'allarme. Se il sistema rileva che l'animale è in stato di fuga, l'interfaccia attiva automaticamente un blocco di sicurezza per impedirne la disattivazione. Inoltre, poiché lo stato dell'allarme si basa sulla variabile condivisa `_isUpdatingAlarm`, l'integrità dell'operazione è garantita da un **mutex** che congela l'input dell'utente durante la sincronizzazione asincrona con il database. Infine, la schermata raccoglie le statistiche quotidiane come passi, chilometri e tempo di attività dell'animale, permettendo la consultazione dei dati storici tramite un selettore di data.

La visualizzazione dei dettagli delle attività di una giornata prosegue in `daily_recap.dart`, che raccoglie e organizza in ordine cronologico ogni monitoraggio visualizzato nella dashboard, con la possibilità di filtrare per specifiche attività. Selezionando una determinata sessione dal riepilogo, si accede alla schermata `activity_details.dart`, che mostra il tracciato specifico del percorso tramite una scia continua sulla mappa. Il sistema utilizza

la funzione **CameraFit** per centrare l'inquadratura all'apertura, garantendo l'immediata visibilità dell'intero tragitto. Inoltre, la schermata ripropone le statistiche di passi, chilometri e durata, ricalcolate specificamente per l'attività selezionata.



(a) Visualizzazione di tutte le attività.



(b) Dettagli di un'attività.

4.6.2 Mappe in Tempo Reale

L'app utilizza mappe interattive sviluppate con `flutter_map`, permettendo di localizzare utente e animale in una visuale satellitare o stradale. La libreria viene ottimizzata tramite l'uso dei tile di `OpenStreetMap`, ovvero piccoli tasselli d'immagine caricati dinamicamente solo per l'area inquadrata, riducendo drasticamente il consumo di dati. Per preservare la batteria dello smartphone, le posizioni si aggiornano in tempo reale solo quando c'è un vero movimento, inoltre l'utente viene localizzato solo se ha dato il permesso.

Il sistema di controllo geografico si articola in due schermate principali che si scambiano dinamicamente in base allo stato dell'allarme. Quando l'allarme è disattivato, l'utente accede al sistema di `geofencing.dart`, un robusto strumento di sicurezza preventiva che permette di definire e gestire i confini virtuali rappresentati come poligoni sulla mappa.

All'attivazione dell'allarme, la schermata `geofencing.dart` passa a `tracking.dart`. In questa fase, il sistema adotta un approccio proattivo di sicurezza implementando un'interfaccia adattiva che cambia tra una modalità di monitoraggio e una di "inseguimento" qualora l'animale lasci la sua zona sicura attiva. Sulla mappa inoltre viene renderizzata una scia storica delle ultime posizioni registrate, fornendo un percorso visivo immediato che aiuta a comprendere la direzione del movimento recente dell'animale.

4.6.3 Gestione delle Zone Sicure

Dopo l'inserimento di un indirizzo (manuale o GPS) o la selezione di una zona dalla pagina di geofencing, la creazione e modifica delle aree avviene tramite `polygon_editor.dart`. L'utente può definire il perimetro toccando lo schermo per aggiungere i vertici. L'interfaccia è progettata per essere estremamente flessibile, consentendo all'utente non solo di aggiungere o annullare i vertici, ma anche di rifinirne la forma trascinando i punti già

posizionati. Oltre a richiedere un minimo di tre vertici per formare una figura geometrica chiusa, un'area può essere creata solo se rispetta due controlli di validità:

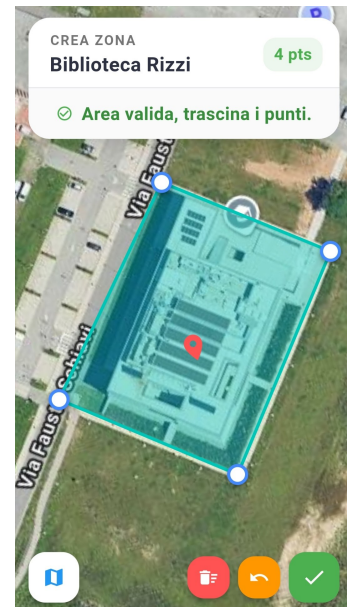
- Il **baricentro dell'area** non deve essere troppo distante dall'indirizzo selezionato, impedendo così la creazione di zone a più di 50 metri dal segnaposto reale.
- Non deve esserci alcuna **sovrapposizione con zone già esistenti** al fine di garantire l'univocità dei perimetri ed evitare conflitti logici durante il tracciamento.



(a) Scelta dell'inserimento.



(b) Compilazione indirizzo.



(c) Disegno dell'area.

5 Sistema di Notifiche Push

L'implementazione di un sistema di notifiche push è fondamentale per un dispositivo di tracciamento animali, in quanto permette di avvisare tempestivamente l'utente in caso di pericoli (uscita dal perimetro sicuro) o problemi tecnici (batteria scarica).

5.1 Analisi dei Vincoli Tecnologici: Incompatibilità Goja e Firebase

Durante la fase di sviluppo, è emerso un vincolo tecnico significativo riguardante l'integrazione tra il backend PocketBase e l'SDK ufficiale di Firebase. PocketBase è scritto in linguaggio Go e utilizza **Goja** come motore di esecuzione JavaScript per la gestione della logica server-side (hooks). Goja è un'implementazione di ECMAScript 5.1 che presenta limitazioni strutturali rispetto a un ambiente Node.js [12] standard:

- **Mancanza di moduli nativi:** Goja non supporta i moduli core di Node.js (come `fs`, `crypto` o `net`), indispensabili per il funzionamento dell'SDK di Firebase.
- **Runtime isolato:** L'SDK `firebase-admin` [13] richiede un ambiente di esecuzione completo per gestire l'autenticazione tramite Service Account e le connessioni persistenti ai server Google.

Per superare queste limitazioni, è stata adottata un'architettura a **Bridge**, separando la logica di business dall'invio fisico dei messaggi.

5.2 Architettura del FCM Bridge

Il sistema si basa su un microservizio esterno sviluppato in Node.js [12], denominato *FCM Bridge*. L'interazione avviene secondo il seguente flusso:

1. **PocketBase (Goja)**: Rileva un evento (es. batteria scarica) e invia una richiesta HTTP POST al bridge tramite il modulo interno `$http.send`.
2. **FCM Bridge (Node.js)**: Riceve la richiesta, carica il Service Account di Firebase e inoltra la notifica tramite l'SDK ufficiale [13].
3. **Feedback e pulizia token**: Il bridge restituisce lo stato della consegna. In caso di errore 404 (token non registrato, app disinstallata) o 400 (token malformato), il backend provvede alla rimozione automatica del token dal profilo utente. I token FCM sono memorizzati come array JSON per supportare più dispositivi per singolo utente.

5.3 Logica di Invio e Trigger

Le notifiche vengono scatenate dai seguenti eventi, ciascuno con logica contestuale per evitare invii ridondanti:

Evento	Condizione di trigger	Nota
Uscita dalla zona	$i \rightarrow s$ oppure $i \rightarrow w$	Dipende dalla presenza di allarme utente
Rientro in zona	$s \rightarrow i$ oppure $w \rightarrow i$	
Allarme da passeggiata	$w \rightarrow s$	Allarme riattivato mentre fuori zona
Inizio viaggio confermato	qualsiasi $\rightarrow v/r$	Dipende dallo stato dell'allarme
Fine viaggio in zona	$v/r \rightarrow i$	
Fine viaggio fuori zona (allarme)	$v/r \rightarrow s$	
Fine viaggio fuori zona (libero)	$v/r \rightarrow w$	
Nessuna geofence	$i \text{ o } s \rightarrow w$	Zone disattivate
Batteria bassa	$\leq 20\%$ (cambio soglia)	Solo al cambio di fascia
Batteria critica	$\leq 10\%$ (cambio soglia)	Solo al cambio di fascia
Batteria in carica	<code>charging</code> passa a <code>true</code>	Solo al cambio di stato
Watchdog	Silenzi > 10 minuti	Chiusura automatica sessione

Tabella 2: Trigger del sistema di notifiche push

5.4 Gestione permessi

Per abilitare la ricezione delle notifiche lato client, è stato fondamentale integrare il file di configurazione `google-services.json`, generato direttamente dalla console di Firebase

e inserito all'interno della directory `android/app` del codice sorgente. Questo file contiene le chiavi API, l'identificativo del progetto (`Project ID`) e i dettagli di configurazione necessari per autenticare in modo sicuro l'applicazione mobile con i servizi Google, senza esporre le credenziali nel codice in chiaro. Durante la fase di build, il *plugin Gradle* di Google Services, che si occupa di mappare i parametri di configurazione nel codice Android, processa questo file per inizializzare automaticamente l'SDK di Firebase all'avvio dell'app, abilitando così la connessione sicura a FCM e la generazione del token univoco del dispositivo. Quest'ultimo viene dapprima salvato localmente e poi sincronizzato con il profilo utente su PocketBase all'interno del campo `tokenFCM`. Tale campo è modellato come una lista per permettere all'utente di ricevere le allerte su più dispositivi contemporaneamente; prima di ogni salvataggio, un controllo di duplicazione verifica che il *token* non sia già presente, evitando così la ricezione di notifiche ridondanti.

6 Vincoli di Sistema

Nonostante i risultati ottenuti, il sistema presenta alcune limitazioni:

- **Dipendenza dalla rete LTE:** senza copertura la trasmissione dati non è garantita.
- **Prestazioni indoor:** il modulo GNSS risulta poco affidabile in ambienti chiusi.
- **Backend locale:** l'utilizzo di PocketBase su macchina locale, esposto tramite ngrok, introduce dipendenze da servizi esterni e limita la scalabilità.
- **Race condition sui pacchetti simultanei:** in caso di ricezione quasi simultanea di due pacchetti dallo stesso dispositivo, entrambi potrebbero leggere lo stesso record `activity` prima del completamento della prima scrittura. Gli scenari più critici sono due: nel caso di cambio di stato, il secondo pacchetto potrebbe sovrascrivere un record già chiuso dal primo, generando inconsistenze strutturali; nel caso di risveglio simultaneo dal *deep sleep*, entrambi i pacchetti potrebbero riattivare lo stesso record `sleep` e creare due sessioni attive contemporaneamente, violando l'invariante del sistema. L'impossibilità di gestire transazioni atomiche isolate sui singoli record tramite le API standard di PocketBase rende questo scenario non eliminabile lato applicativo. Tuttavia, la frequenza di invio (ogni 30 o più secondi) rende l'evento estremamente raro all'atto pratico.
- **Fallimento nella geolocalizzazione:** l'eventuale mancato rilevamento di indirizzi, solitamente nei centri abitati più piccoli, non è imputabile al codice, bensì a un limite legato alla densità dei metadati geografici. Poiché `OpenStreetMap` si basa su contributi volontari, l'assenza di tag specifici (come i numeri civici o il CAP associato alla via) rende inefficaci le query strutturate inviate a `Nominatim`. Quest'ultimo, richiedendo una corrispondenza rigida ed esatta di tutti i parametri forniti, non è in grado di compensare l'incompletezza dei dati.

7 Implementazioni Future

Il progetto nel suo percorso di sviluppo ha trovato delle possibilità future di scalabilità dell'App, di seguito le idee che abbiamo accavallato per procedere nella migliore implementazione delle precedenti funzioni descritte.

7.1 Stato della batteria

Per garantire all'utente un corretto valore della batteria aggiornato sugli ultimi pacchetti è richiesto un calcolo con dati storici, questo è uno dei possibili ulteriori passaggi che può essere sviluppato all'interno dell'applicazione per quanto riguarda i dati da fornire all'utente. Non risulta come attività critica in quanto il sistema riesce a garantire all'utente una notifica nel momento in cui la batteria raggiunge (e scende) sotto il livello del 20%.

7.2 Database, Infrastruttura Cloud e Gestione della Concorrenza

Il passaggio da un ambiente di prototipazione locale a una soluzione orientata alla produzione prevede la migrazione dell'intero backend su un'infrastruttura Cloud dedicata (PaaS o VPS come AWS, Google Cloud o DigitalOcean). Questa evoluzione consentirà di superare i vincoli attuali attraverso i seguenti interventi strutturali:

- **Assegnazione di Dominio e Sicurezza TLS:** Sostituendo il tunnel temporaneo `ngrok` con un dominio di secondo livello proprietario, protetto da certificati SSL/TLS (es. Let's Encrypt), si garantirà una connessione HTTPS/WSS nativa, stabile e priva di interruzioni di sessione o rate-limiting arbitrari.
- **Risoluzione delle Race Condition via Message Broker:** Per azzerare il rischio di collisione nell'aggiornamento simultaneo delle attività (derivante dalla ricezione di pacchetti ravvicinati), si prevede l'introduzione di un middleware di messaggistica asincrona, come `Redis` o `RabbitMQ`, interposto tra l'endpoint di ricezione e il database. Questo livello fungerà da coda FIFO (*First-In, First-Out*), serializzando i pacchetti provenienti dallo stesso tracker ed eseguendo le transazioni in modo strettamente sequenziale, preservando così l'invariante del sistema.
- **Migrazione del DBMS:** Qualora il volume di dispositivi connessi dovesse saturare le capacità di lock di SQLite, l'architettura consentirà di scalare verticalmente migrando verso un database relazionale centralizzato (come `PostgreSQL`), nativamente ottimizzato per transazioni ACID altamente concorrenti.

7.3 Ingegnerizzazione e Costruzione della Scheda Hardware

L'attuale prototipo hardware, sebbene funzionale alla validazione del software, richiede una fase di ingegnerizzazione elettronica per poter essere trasformato in un dispositivo commerciale realmente indossabile dall'animale. Lo sviluppo futuro si articolerà su tre assi principali:

- **Sviluppo di un PCB Custom:** Progettazione di un circuito stampato personalizzato (*Printed Circuit Board*) mediante strumenti EDA (come KiCad o Altium Designer). L'integrazione del microcontrollore, del modem LTE e del modulo GNSS su un'unica scheda consentirà di ridurre drasticamente il fattore di forma (*form factor*), minimizzando l'ingombro e il peso del tracker sul collare dell'animale.
- **Ottimizzazione del Power Management:** Introduzione di un circuito integrato dedicato alla gestione dell'energia (*PMIC*) e di un chip di *fuel gauge* hardware (es. serie MAX17043). Questo permetterà di implementare il calcolo predittivo basato sui dati storici menzionato nella sezione precedente, offrendo letture della

percentuale di carica estremamente precise e non inficiate dai picchi di assorbimento del modem durante la trasmissione.

- **Industrial Design e Protezione Ambientale:** Progettazione di un *enclosure* (involucro) personalizzato tramite modellazione 3D e stampaggio a iniezione. La scocca dovrà garantire un grado di protezione minimo pari a **IP67** o **IP68**, rendendo il dispositivo totalmente impermeabile all'acqua, al fango e resistente agli urti causati dai movimenti repentini dell'animale all'aperto.

8 Conclusioni

Riferimenti bibliografici

- [1] *T-SIM7670G-S3 Specifications*. URL: <https://github.com/Xinyuan-LilyGO/LilyGo-Modem-Series/blob/main/docs/en/esp32s3/sim7670g-s3/REAMDE.MD>.
- [2] *LTE Antenna Specifications*. URL: <https://github.com/Xinyuan-LilyGO/LilyGo-Modem-Series/blob/main/datasheet/LTE%20Antenna%20Specifications.pdf>.
- [3] *GPS Antenna Specifications*. URL: <https://github.com/Xinyuan-LilyGO/LilyGo-Modem-Series/blob/main/datasheet/GPS%20Antenna%20Specifications.pdf>.
- [4] *BMA400 CJMCU-400 Datasheet*. URL: https://www.bosch-sensortec.com/media/boschsensortec/downloads/product_flyer/bst-bma400-fl000.pdf.
- [5] *PocketBase Documentation*. URL: <https://pocketbase.io/docs/>.
- [6] *ngrok Documentation - HTTP Tunnels*. URL: <https://ngrok.com/docs/secure-tunnels/>.
- [7] *rclone - rsync for cloud storage*. URL: <https://rclone.org/>.
- [8] *Backblaze B2 Cloud Storage*. URL: <https://www.backblaze.com/b2/cloud-storage.html>.
- [9] *UptimeRobot - Free Website Monitoring Service*. URL: <https://uptimerobot.com/>.
- [10] *Flutter Documentation*. URL: <https://docs.flutter.dev/>.
- [11] *Nominatim Documentation*. URL: <https://nominatim.org/release-docs/latest/>.
- [12] *Node.js Documentation*. URL: <https://nodejs.org/en/docs/>.
- [13] *Firebase Admin SDK - Cloud Messaging*. URL: <https://firebase.google.com/docs/cloud-messaging/server>.